

MentalOS — Dokumentacja

Zespół: X
Data: 2026-05-19
Projekt: MentalOS

Spis treści

- 1. Opis projektu
- 2. Stos technologiczny
- 3. Architektura systemu
- 4. Struktura katalogów
- 5. Backend — warstwa danych (Domain)
- 6. Backend — baza danych i EF Core
- 7. Backend — serwisy (Services)
- 8. Backend — kontrolery i endpointy API
- 9. Backend — middleware, DTOs i konfiguracja startowa
- 10. Frontend — architektura i nawigacja
- 11. Frontend — moduły funkcjonalne
- 12. Frontend — zarządzanie stanem
- 13. Frontend — warstwa API i komunikacja
- 14. Schemat bazy danych
- 15. Uwierzytelnienie i autoryzacja
- 16. Konfiguracja i zmienne środowiskowe
- 17. Przepływy danych — kluczowe scenariusze
- 18. Instalacja i uruchomienie
- 19. Logowanie i monitoring
- 20. Bezpieczeństwo
- 21. Znane ograniczenia i plany rozwoju

1. Opis projektu

MentalOS to mobilna aplikacja do monitorowania zdrowia psychicznego, tworzona przez zespół studentów w ramach przedmiotu Programowanie Zespołowe. Aplikacja wspiera użytkowników w codziennej dbałości o dobrostan psychiczny poprzez szereg wzajemnie powiązanych funkcjonalności.

Główne funkcjonalności

Obszar	Opis
Dziennik	Prowadzenie wpisów z oceną nastroju, tagowaniem emocji i generowaniem podsumowań przez AI
Asystent AI	Kontekstowy czat z modelem językowym świadomym historii użytkownika

Obszar	Opis
Planer	Zarządzanie zadaniami z przypomnieniami, kategoriami i cyklicznością
Wirtualny ogród	Mechanika gamifikacyjna — wzrost roślin w 5 stadiach powiązany z aktywnością
Sklep i Akcesoria	System walut (monety, owoce) i nagrody za aktywność, zmiana wyglądu chmurki
Mental Support	Materiały do medytacji, ćwiczenia oddechowe, dźwięki natury, filmy szkoleniowe
Test osobowości	Dwa niezależne systemy: quiz typologiczny (4 typy) + Big Five OCEAN
Wyrocznia	Interaktywna chmurka-wróżbita na ekranie głównym (animacja z odpowiedzią Tak/Nie)
Profil	Zarządzanie kontem, awatarem, subskrypcją premium
Panel admina	Zarządzanie użytkownikami, podgląd logów, zarządzanie sklepem (Blazor SSR)

2. Stos technologiczny

Backend

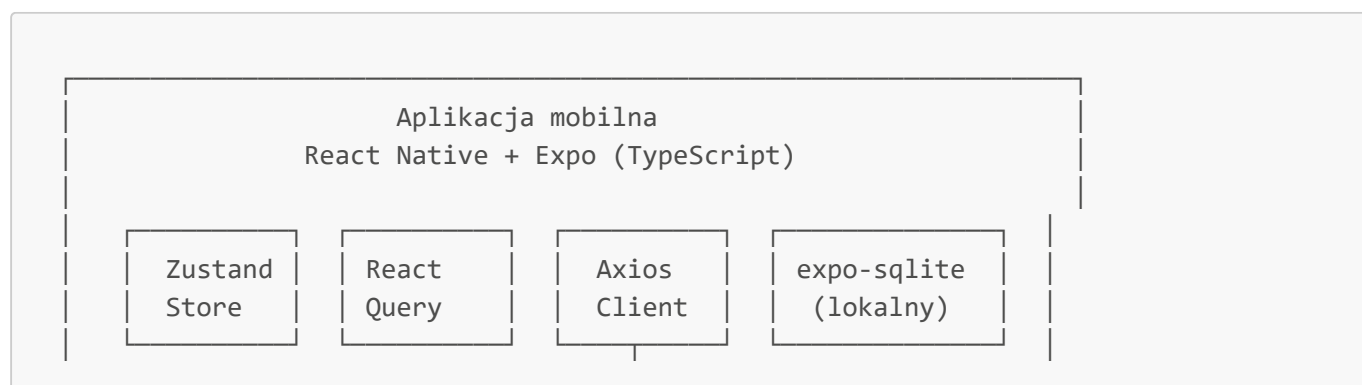
Komponent	Technologia	Wersja
Framework	ASP.NET Core Web API	8.0
Język	C#	12.0
ORM	Entity Framework Core	8.0.11
Baza danych	PostgreSQL	16+
Provider DB	Npgsql.EF Core	8.0.11
Uwierzytelnienie	JWT Bearer	8.0.11
OAuth	Google.Apis.Auth	1.69.0
Logowanie	Serilog.AspNetCore	8.0.3
Dokumentacja API	Swashbuckle (Swagger)	6.5.0
Email	MailKit / MimeKit	4.15.1
Panel admina	Blazor SSR (Interactive Server)	—
Konteneryzacja	Docker	—
Haszowanie haseł	ASP.NET Core Identity PasswordHasher	—
AI Chat	OpenAI API (GPT-4o-mini)	—
AI Transkrypcja	OpenAI Whisper (gpt-4o-mini-transcribe)	—

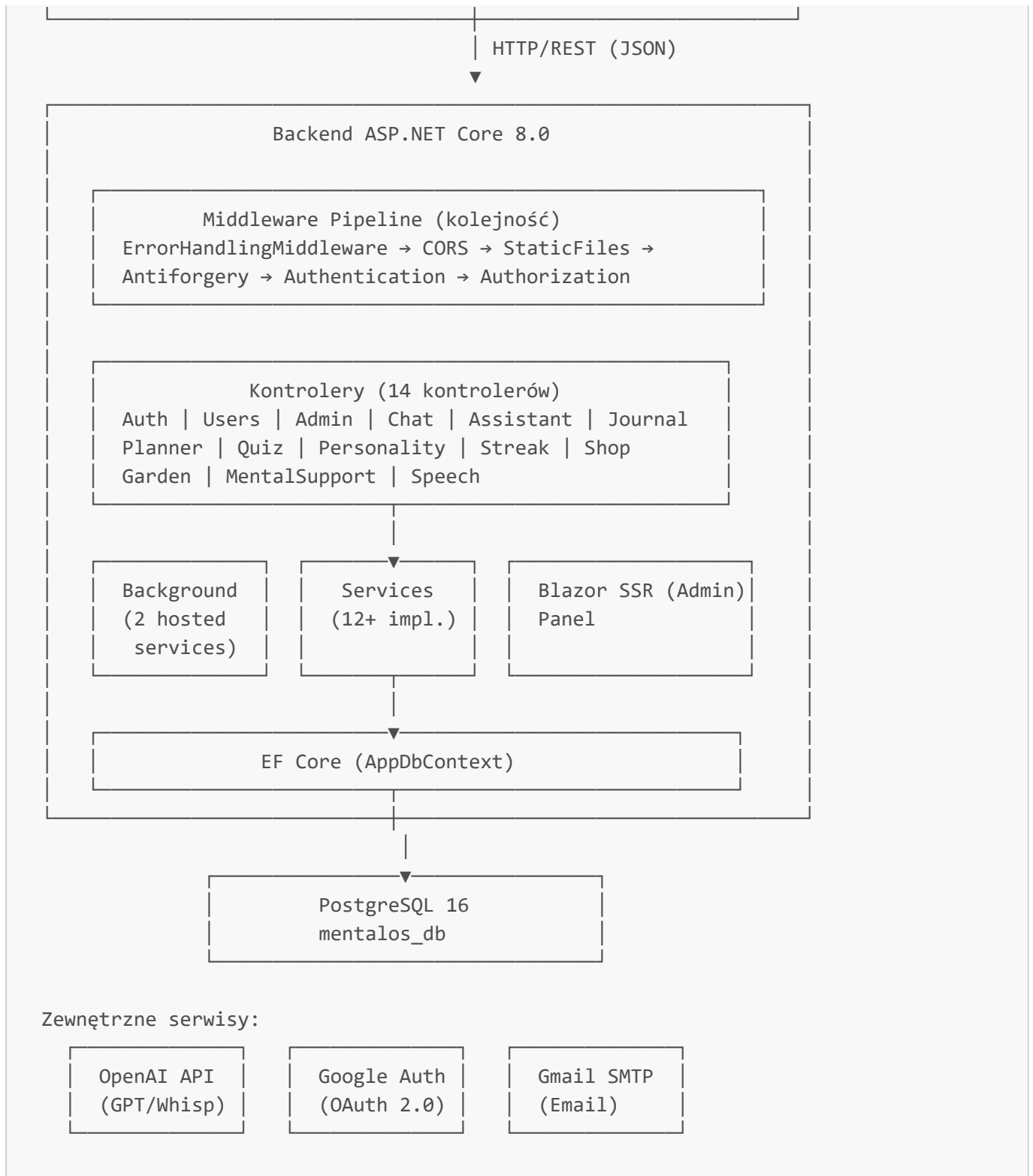
Frontend

Komponent	Technologia	Wersja
Framework	React Native	0.81.5
Platforma	Expo	~54.0.31
Język	TypeScript	~5.9.2
Routing	Expo Router	~6.0.21
Stan	Zustand	^5.0.11
HTTP	Axios	^1.13.6
Cache/Query	TanStack React Query	^5.90.21
Lokalna baza	SQLite (expo-sqlite)	~16.0.10
Bezpieczny storage	expo-secure-store	~15.0.8
Animacje	react-native-reanimated	~4.1.1
Gesty	react-native-gesture-handler	~2.28.0
Nawigacja dolna	@react-navigation/bottom-tabs	^7.4.0
Kalendarz	react-native-calendars	^1.1314.0
Edytor tekstu	react-native-pell-rich-editor	^1.10.0
Google OAuth	@react-native-google-signin	^16.1.2
Dekodowanie JWT	jwt-decode	^4.0.0
Audio	expo-audio / expo-av	~1.1.1 / ~16.0.8
Wideo	expo-video	~3.0.16
Aparat / media	expo-image-picker	~17.0.10
Powiadomienia	expo-notifications	~0.32.16

3. Architektura systemu

Diagram ogólny





Wzorce architektoniczne

Backend stosuje **architekturę warstwową**:

```
Controllers (warstwa prezentacji / API)
|
Services (warstwa logiki biznesowej)
|
Data / EF Core (warstwa dostępu do danych)
|
```

```

Domain (warstwa modeli dziedzinowych)
|
Middleware (aspekty przekrojowe: obsługa błędów, logowanie)

```

Zastosowane wzorce projektowe:

- **Dependency Injection** — wbudowane w ASP.NET Core, wszystkie serwisy rejestrowane w `Program.cs`
- **Repository Pattern** — realizowany poprzez `AppDbContext` i EF Core
- **Service Layer Pattern** — oddzielenie logiki biznesowej od kontrolerów
- **Middleware Pattern** — globalna obsługa błędów, pipeline żądań
- **Interface Segregation** — każdy serwis ma dedykowany interfejs w `Services/Interfaces/`
- **Hosted Services** — zadania w tle jako `IHostedService`
- **Options Pattern** — konfiguracja OpenAI przez `IOptions<OpenAiOptions>`

4. Struktura katalogów

Backend

```

backend/
├── MentalOS/
│   ├── Program.cs                # Punkt wejścia, konfiguracja DI i
middleware
│   ├── MentalOS.csproj           # Definicja projektu i paczki NuGet
│   ├── appsettings.json          # Konfiguracja (connection string,
JWT, OAuth, AI, SMTP)
│   ├── appsettings.Development.json # Konfiguracja środowiska
deweloperskiego
│   ├── Dockerfile                # Konteneryzacja Docker
│   ├── Controllers/              # 13 kontrolerów API
│   │   ├── AuthController.cs     # [POST] register, login, google,
facebook, forgot-password, reset-password
│   │   ├── UsersController.cs    # [GET/PUT/DELETE] me, premium, avatar
upload
│   │   ├── AdminController.cs    # [admin] users CRUD, logs, add-item
│   │   └── JournalController.cs  # Dziennik: CRUD, AI summary, AI entry
z transkrypcji
│   │   └── ChatController.cs      # Sesje czatu, wiadomości, historia,
debug-context
│   │   └── AssistantController.cs # Uproszczony interfejs czatu (auto-
wybór ostatniej sesji)
│   │   └── PlannerController.cs   # Zadania: CRUD, daily/weekly/monthly,
toggle-complete
│   │   └── QuizController.cs      # Quiz typologiczny (8 pytań, 4 typy
osobowości)
│   │   └── PersonalityController.cs # Big Five OCEAN (PersonalityService +
JsonQuestionProvider)
│   │   └── StreakController.cs    # Streak, daily-status, nagrody
dzienny, owoce

```

ShopController.cs	# Sklep: items, buy, equip, unequip,
my-items, history	
GardenController.cs	# Ogród: status, plant, harvest,
exchange-fruit	
MentalSupportController.cs	# video-watched (welcome reward za
film)	
SpeechController.cs	# Transkrypcja audio (multipart/form-
data → Whisper)	
Domain/	# Encje bazy danych (modele
dziedziczne)	
User.cs	# Konto + gamifikacja + pola audytowe
Role.cs + UserRole (w Role.cs)	# Role i tabela łącząca
PersonalityProfile.cs	# Profil OCEAN/typologiczny (JSON
traits)	
PersonalityQuestion.cs	# Pytania Big Five (dla
JsonQuestionProvider)	
JournalEntry.cs	# Wpisy dziennika (z preview,
isSummary)	
ChatSession.cs + ChatMessage.cs	# Sesje i wiadomości AI
PlannerTask.cs	# Zadania z enumami Priority i
Recurrence	
StreakHistory.cs	# Historia zdarzeń streak
CoinTransaction.cs	# Transakcje monet
ShopItem.cs	# Katalog sklepu (+
FrontendAccessoriesId)	
UserItem.cs	# Zakupione przedmioty użytkownika
Garden.cs + GardenBed.cs	# Ogród (enum TreeState: 5 stanów)
PasswordResetToken.cs	# Tokeny resetu hasła
Data/	
AppDbContext.cs	# EF Core DbContext – wszystkie
DbSet<T>	
Services/	
TokenService.cs	# JWT – generowanie tokenów
OAuthService.cs	# Google + Facebook OAuth
ChatService.cs	# Sesje czatu,
MAX_HISTORY_MESSAGES=20, system prompt	
OpenAiChatService.cs	# Integracja z OpenAI Chat API (GPT-
4o-mini)	
OpenAiSpeechService.cs	# Transkrypcja audio (Whisper)
ContextBuilder.cs	# Budowanie kontekstu AI (z
anonimizacją danych)	
GardenService.cs	# Cykl życia roślin, FastMode, siatka
2x3	
StreakService.cs	# Streak narastający (coins =
streak_count)	
ShopService.cs	# Zakupy, equip/unequip, transakcje
PersonalityService.cs	# Obliczanie Big Five OCEAN
JsonQuestionProvider.cs	# Pytania quizu z pliku JSON
PasswordResetService.cs	# Reset hasła: SHA-256 token, 24h,
jednorazowy	
GmailSmtpEmailService.cs	# Email przez Gmail SMTP (MailKit)

```

|   |   | WelcomeRewardService.cs          # Jednorazowe nagrody: note=10,
task=5, video=10, quiz=50
|   |   | DailySummaryNotificationService.cs # Hosted service: powiadomienia
push
|   |   | DataArchivingService.cs          # Hosted service: archiwizacja starych
danych
|   |   | └ Interfaces/                    # Interfejsy serwisów (13 plików)
|   |   |
|   |   | └ DTOs/                          # Data Transfer Objects
|   |   |   | └ UserDataDto.cs + UpdateUserDataDto.cs
|   |   |   | └ WelcomeRewardDto.cs        # { Granted, CoinsAwarded, RewardType
}
|   |   |   | └ ForgotPasswordDto.cs
|   |   |   | └ GardenStatusDto.cs + GardenBedDto.cs
|   |   |   | └ CreateShopItemDto.cs
|   |   |   | └ TranscriptionResultDto.cs
|   |   |   | └ ChatDTOs/ (ChatRequestDto, ChatMessageDto, ChatResponseDto)
|   |   |   | └ JournalDTOs/ (JournalEntryDto, CreateJournalEntryDto,
UpdateJournalEntryDto,
|   |   |   |   | GenerateSummaryRequestDto, GenerateAiEntryRequestDto)
|   |   |   |   | └ PlannerDTOs/ (PlannerTaskDto, CreatePlannerTaskDto,
UpdatePlannerTaskDto)
|   |   |   |   | └ QuizDTOs/ (QuizQuestionDto, QuizOptionDto, QuizResultDto,
SubmitQuizDto)
|   |   |   |
|   |   |   | └ Middleware/
|   |   |   |   | └ ErrorHandlingMiddleware.cs    # Globalna obsługa wyjątków → JSON 500
|   |   |   |
|   |   |   | └ Options/
|   |   |   |   | └ OpenAiOptions.cs             # Bind do sekcji "OpenAI" w
appsettings
|   |   |   |
|   |   |   | └ Migrations/                     # 8 migracji EF Core (od 2026-04-28)
|   |   |   |
|   |   |   | └ Components/                     # Blazor SSR (panel admina)
|   |   |   |   | └ App.razor
|   |   |   |
|   |   |   | └ Properties/
|   |   |   |   | └ launchSettings.json          # Porty: HTTP 5076, HTTPS 5078
|   |   |   |
|   |   |   | └ Logs/                          # Serilog – log-YYYY-MM-DD.txt

```

Frontend

```

frontend/
├─ app.json          # Konfiguracja Expo
├─ package.json      # Zależności npm
├─ tsconfig.json     # Konfiguracja TypeScript
├─
├─ app/              # Routing – Expo Router (file-based)
├─   └─ _layout.tsx  # Root layout: QueryClient, AppInit,

```

```

Toast, WelcomeRewardModal
|   |— index.tsx                # Guard nawigacyjny (isAuthenticated →
home lub login)
|   |— login.tsx                # Ekran logowania
|   |— register.tsx            # Ekran rejestracji
|   |— onboarding.tsx          # Onboarding (konfiguracja)
preferencji)
|   |— psychotype.tsx          # Test osobowości (quiz typologiczny)
|   |— garden.tsx              # Wirtualny ogród
|   |— accessories.tsx         # Sklep i akcesoria
|   |— modal.tsx               # Globalny ekran modalny
|   |— (tabs)/                 # Główna nawigacja dolna (5 zakładek)
|   |   |— _layout.tsx         # Konfiguracja bottom tab bar
|   |   |— home.tsx            # Ekran główny: Drzewo Dnia,
Wyrocznia, premium
|   |   |— planer.tsx           # Planer zadań
|   |   |— profile.tsx         # Profil użytkownika (re-eksport)
|   |   |— assistant.tsx       # Asystent AI (lista sesji)
|   |   |— logout.tsx          # Wylogowanie
|   |   |— diary/
|   |   |   |— _layout.tsx
|   |   |   |— index.tsx
|   |   |   |— entry.tsx
|   |   |   |— note.tsx
|   |   |   |— test.tsx
|   |   |— mentalSupport/
|   |   |   |— _layout.tsx
|   |   |   |— index.tsx
|   |   |   |— breathing.tsx
|   |   |   |— meditation.tsx
|   |   |   |— nature.tsx
|   |   |   |— training.tsx
|   |   |   |— video/[id].tsx
|   |   |— psychologists/
|   |   |   |— index.tsx
|   |   |   |— [id].tsx
|   |   |— visits/
|   |   |   |— index.tsx
|   |— src/
|   |   |— hooks/
|   |   |   |— useAuthMutations.ts    # Mutacje logowania/rejestracji (React
Query)
|   |   |   |— useNetworkStatus.ts    # @react-native-community/netinfo
|   |   |   |— useUser.ts             # Pobieranie danych zalogowanego
użytkownika
|   |   |   |— use-color-scheme.ts    # Motyw jasny/ciemny
|   |   |— services/
|   |   |   |— api/
|   |   |       |— client.ts          # Bazowy klient Axios + interceptory
JWT
|   |   |   |— auth.ts                # login, register, googleLogin,
facebookLogin

```



```

| | | | └─ diaryApi.ts # getEntries, createEntry,
updateEntry, deleteEntry
| | | | └─ streakApi.ts # getDailyStatus, claimDailyReward,
claimFruit, triggerDaily, trackVideoWatched
| | | | └─ assistant.ts # createSession, sendMessage,
getHistory
| | | | └─ chatApi.ts # (alias)
| | | | └─ store/
| | | | └─ useAuthStore.ts # Token, user, isAuthenticated,
hydrate (z offline grace)
| | | | └─ useShopStore.ts # Stan sklepu, inwentarz,
equippedPreviewImage
| | | | └─ useVisitStore.ts # Historia wizyt (ładowana per userId)
| | | | └─ useRewardModalStore.ts # Stan modalu WelcomeReward
| | | | └─ useToastStore.ts # Globalne powiadomienia Toast
| | | | └─ settingsStorage.ts # Ustawienia w AsyncStorage
| | | | └─ storage.ts # Wrapper:
saveToken/getToken/clearToken/saveUser/getUser
| | | └─ network/networkUtils.ts # canReachBackend() – sprawdzenie
dostępności serwera
| | | └─ modules/
| | | └─ diary/
| | | └─ db/diaryDb.ts # Inicjalizacja SQLite (diary.db),
schemat tabeli
| | | | └─ services/diaryService.ts # CRUD na SQLite (getAll, create,
update, delete)
| | | | └─ services/diarySyncService.ts # Synchronizacja pending→serwer
| | | | └─ services/testResultTransfer.ts # Transfer wyników testu dziennego
| | | | └─ hooks/useDiaryEntries.ts # Hook: łączy diaryService z
komponentami
| | | | └─ components/ # MoodSelector, TagSelector,
SummaryInput, DiaryEntryCard...
| | | | └─ screens/ # DiaryScreen, DiaryEntryScreen,
DiaryNoteScreen, DiaryTestScreen
| | | | └─ diary.types.ts # Typ DiaryEntry z syncStatus
| | | | └─ assistant/ # ChatScreen, MessageBubble,
PersonalityDrawer
| | | | └─ garden/ # GardenBoard, GardenSlot, GardenTree
| | | | └─ mentalSupport/ # Medytacje, Breathing, Nature,
Training
| | | | └─ planner/ # PlannerScreen, PlannerTaskCard,
PlannerInputBar
| | | | └─ shop/ # ShopScreen, ItemCard, BalanceBar
| | | | └─ profile/screens/ProfileScreen.tsx
| | | | └─ psychotype/ # QuizScreen, ResultScreen
| | | | └─ psychologists/ # Lista i profil psychologa
| | | | └─ visits/ # Historia wizyt
| | | └─ shared/
| | | | └─ components/ # Współdzielone komponenty UI
| | | | └─ layout/LayoutContainer.tsx # Wrapper layoutu ekranu
| | | | └─ theme/colors.ts # Paleta kolorów
| | | | └─ theme/typography.ts # Style typografii

```

```

|   |   |— theme/spacing.ts           # Stałe odstępów
|   |   |— theme/styles.ts           # cardStyles i inne wspólne style
|   |   |— constants/                # Stałe aplikacji
|   |— types/auth.types.ts           # UserPayload (dekodowany z JWT)
|— assets/
|   |— garden/                       # tree-stage-0.png, tree-stage-1.png,
tree-stage-2.png
|   |— images/                       # cloud.png, fruit.png,
oracleBubble.png

```

5. Backend — warstwa danych (Domain)

User

Główna encja systemu — konto użytkownika z danymi autoryzacji, stanem gamifikacyjnym i pełnymi polami audytowymi.

```

[Table("users")]
public class User
{
    // Identyfikacja
    [Column("id")]           public Guid Id { get; set; }
    [Column("email")]        public string Email { get; set; }
    [Column("password_hash")] public string? PasswordHash { get; set; }
    [Column("first_name")]   public string? FirstName { get; set; }
    [Column("last_name")]    public string? LastName { get; set; }
    [Column("avatar")]       public string? Avatar { get; set; }

    // Streak (seria)
    [Column("streak_count")] public int StreakCount { get; set; } = 0;
    [Column("streak_active")] public bool StreakActive { get; set; } =
false;
    [Column("last_activity_date")] public DateTime? LastActivityDate { get; set; }
}

// Waluta
[Column("coins_balance")] public int CoinsBalance { get; set; } = 0;
[Column("fruits_balance")] public int FruitsBalance { get; set; } = 0;
[Column("has_pending_fruit")] public bool HasPendingFruit { get; set; } =
false;
[Column("is_premium")]     public bool IsPremium { get; set; } = false;

// Flagi jednorazowych nagród (WelcomeReward)
[Column("has_received_note_reward")] public bool HasReceivedNoteReward { get;
set; } = false;
[Column("has_received_task_reward")] public bool HasReceivedTaskReward { get;
set; } = false;
[Column("has_received_video_reward")] public bool HasReceivedVideoReward {

```

```

get; set; } = false;
[Column("has_received_quiz_reward")] public bool HasReceivedQuizReward { get;
set; } = false;

// Pola audytowe (pełne śledzenie zmian)
[Column("created_at")] public DateTime CreatedAt { get; set; }
[Column("created_by")] public Guid? CreatedBy { get; set; }
[Column("updated_at")] public DateTime UpdatedAt { get; set; }
[Column("updated_by")] public Guid? UpdatedBy { get; set; }
[Column("deleted_at")] public DateTime? DeletedAt { get; set; } // soft
delete
[Column("deleted_by")] public Guid? DeletedBy { get; set; }

// OAuth
public string Provider { get; set; } = "local"; // local | google | facebook
public string? ProviderUserId { get; set; }

// Obliczeniowe – nie przechowywane w DB
[NotMapped] public bool IsAdmin { get; set; }
[NotMapped] public string PersonalityType { get; set; } = "unknown";
}

```

Role i UserRole

Relacja many-to-many User ↔ Role przez tabelę łączącą.

```

[Table("roles")]
public class Role { Id, Name (unikalny), Description, pola audytowe }

[Table("user_roles")]
public class UserRole { Id, UserId (FK), RoleId (FK), pola audytowe }

```

Zdefiniowane role: `user`, `admin`, `specialist` (zarezerwowane).

PersonalityProfile

Wynik testu osobowości powiązany 1:1 z użytkownikiem. Przechowuje dane zarówno systemu typologicznego (QuizController), jak i Big Five OCEAN (PersonalityController).

```

[Table("personality_profiles")]
public class PersonalityProfile
{
    public Guid Id, UserId (UNIQUE FK);
    public string PersonalityType; // "empatyk" | "analitik" | "lider" |
    "marzyciel"
                                // lub dominujący wymiar OCEAN: "O" | "C" | "E"
    | "A" | "N"
    public string? Traits; // JSON: { "empatyk": 5, ... } lub { "O": 3.8,
    "C": 4.2, ... }
}

```

```
// + pola audytowe  
}
```

JournalEntry

Wpis w dzienniku — obsługuje zarówno wpisy użytkownika, jak i podsumowania wygenerowane przez AI.

```
[Table("journal_entries")]  
public class JournalEntry  
{  
    Guid Id, UserId;  
    string? Title;  
    string Content;           // pełna treść wpisu  
    int? MoodScore;          // 1-10 (nullable)  
    string? Emotions;         // tagi po przecinku, np. "Spokój,Radość"  
    string? Preview;          // AI-generowane podsumowanie wpisu  
    bool IsSummary;           // true = podsumowanie AI; false = wpis użytkownika  
    DateTime EntryDate, CreatedAt, UpdatedAt;  
    bool IsDeleted;  
    DateTime? ArchivedAt;  
}
```

PlannerTask

Zadanie z pełną obsługą powtarzalności i przypomnień.

```
public enum PlannerTaskPriority { Normal, High }  
public enum PlannerTaskRecurrence { None, Daily, Weekly, Monthly, WorkDays, Custom }  
  
[Table("planner_tasks")]  
public class PlannerTask  
{  
    Guid Id, UserId;  
    string Title;  
    string? Description;  
    DateTime TaskDate;  
    bool HasTime;  
    DateTime? ReminderTime;  
    string? Icon, Category;  
    PlannerTaskPriority Priority;    // domyślnie Normal  
    PlannerTaskRecurrence Recurrence; // domyślnie None  
    bool IsCompleted;  
    DateTime? CompletedAt;  
    DateTime CreatedAt, UpdatedAt;  
    bool IsDeleted;  
    DateTime? ArchivedAt;  
}
```

ChatSession i ChatMessage

```
[Table("chat_sessions")] { Guid Id, UserId; string? Title; DateTime CreatedAt,
UpdatedAt; }
[Table("chat_messages")] { Guid Id, SessionId (FK CASCADE); string Sender; string
Content; DateTime CreatedAt; }
```

ShopItem i UserItem

```
public class ShopItem
{
    Guid Id;
    string FrontendAccesoriesId; // ID używane przez frontend (kolumna
    accesories_id)
                                // umożliwia mapowanie bez znajomości UUID
    string Name, Description;
    int Price;
    string Type;                // "accessory" | inne
    bool IsActive;
    DateTime CreatedAt, UpdatedAt;
}

[Table("user_item")]
public class UserItem { Guid Id, UserId (FK), ShopItemId (FK); DateTime
PurchasedAt; bool IsActive; }
```

Garden i GardenBed

```
[Table("Gardens")] public class Garden { Guid Id, UserId; ICollection<GardenBed>
GardenBeds; }

public enum TreeState { Empty, Sprout, Sapling, Mature, Fruiting } // 5 stanów

public class GardenBed
{
    Guid Id, GardenId;
    TreeState TreeState; // domyślnie Empty
    DateTime? PlantedAt;
    int X, Y;            // pozycja na siatce (2 kolumny x 3 rzędy = 6 pól)
    // UNIQUE: (GardenId, X, Y)
}
```

PasswordResetToken

```
[Table("password_reset_tokens")]
{
    Guid Id, UserId (FK CASCADE);
    string TokenHash; // SHA-256 surowego tokenu, varchar(64), UNIQUE
    DateTime ExpiresAt, CreatedAt;
    DateTime? UsedAt; // null = nieuzyty; timestamp = jednorazowo wykorzystany
}
```

6. Backend — baza danych i EF Core

AppDbContext — zarejestrowane DbSet

DbSet<User>	Users
DbSet<Role>	Roles
DbSet<UserRole>	UserRoles
DbSet<PersonalityProfile>	PersonalityProfiles
DbSet<PersonalityQuestion>	PersonalityQuestions
DbSet<JournalEntry>	JournalEntries
DbSet<ChatSession>	ChatSessions
DbSet<ChatMessage>	ChatMessages
DbSet<PlannerTask>	PlannerTasks
DbSet<StreakHistory>	StreakHistories
DbSet<CoinTransaction>	CoinTransactions
DbSet<ShopItem>	ShopItems
DbSet<UserItem>	UserItems
DbSet<Garden>	Gardens
DbSet<GardenBed>	GardenBeds
DbSet<PasswordResetToken>	PasswordResetTokens

Indeksy i ograniczenia

Tabela	Indeks / Ograniczenie
users	UNIQUE (email)
users	UNIQUE (Provider, ProviderUserId)
roles	UNIQUE (name)
user_roles	UNIQUE (user_id, role_id)
personality_profiles	UNIQUE (user_id)
journal_entries	INDEX (user_id)
planner_tasks	INDEX (user_id), INDEX (task_date)
chat_messages	INDEX (SessionId)

Tabela	Indeks / Ograniczenie
<code>coin_transaction</code>	INDEX (<code>user_id</code>)
<code>streak_history</code>	INDEX (<code>user_id</code>)
<code>GardenBeds</code>	UNIQUE (<code>GardenId</code> , <code>X</code> , <code>Y</code>)
<code>password_reset_tokens</code>	UNIQUE (<code>token_hash</code>), INDEX (<code>user_id</code>)
<code>user_item</code>	UNIQUE (<code>user_id</code> , <code>shop_item_id</code>)

Strategie danych

- **Soft delete:** Rekordy użytkowników i ról nie są trwale usuwane — `deleted_at` timestamp.
- **Pełny cascade:** Tabele `chat_messages`, `coin_transaction`, `journal_entries`, `password_reset_tokens`, `personality_profiles`, `planner_tasks`, `user_item`, `user_roles`, `GardenBeds` mają `ON DELETE CASCADE` do tabeli `users`.
- **Pola audytowe:** `created_at`, `created_by`, `updated_at`, `updated_by`, `deleted_at`, `deleted_by` na kluczowych encjach.
- **JSON w polach tekstowych:** `traits` (PersonalityProfile) i `emotions` (JournalEntry — wartości po przecinku) przechowywane jako tekst.
- **Automatyczna inicjalizacja:** Przy starcie aplikacji EF Core uruchamia migracje i dodaje brakujące kolumny przez `ALTER TABLE ... ADD COLUMN IF NOT EXISTS`.

Migracje EF Core

```
20260428130255_InitialClean
20260505172722_Main
20260505173258_InitialCreate
20260505181517_AddFrontendAccesoriesId
20260505100149_DanyloSOsalo
20260511000000_AddPreviewToJournalEntries
20260511120000_AddFruitsBalanceToUsers
20260512100000_AddHasPendingFruitToUsers
```

```
dotnet ef migrations add <NazwaMigracji> # nowa migracja
dotnet ef database update                # ręczne zastosowanie
dotnet ef migrations list                 # lista migracji
```

7. Backend — serwisy (Services)

TokenService

Generuje tokeny JWT z claims: `NameIdentifier` (User ID), `Email`, `Role` (osobny claim per rola). Algorytm HMAC-SHA256, ważność z konfiguracji (`Jwt:ExpiryMinutes`). Role pobierane z bazy przy generowaniu.

OAuthService

Google: `GoogleJsonWebSignature.ValidateAsync(idToken)` → pobiera `Subject` jako `ProviderUserId`, `Email`, `Name`.

Facebook: `GET https://graph.facebook.com/me?fields=id,name,email&access_token={token}`.

W obu przypadkach: jeśli użytkownik nie istnieje w DB → tworzony z `Provider = "google"/"facebook"` i rolą `"user"`.

ChatService

Zarządza pełnym cyklem konwersacji AI:

- `MAX_HISTORY_MESSAGES = 20` — ostatnie 20 wiadomości dołączane do kontekstu
- System prompt: `"You are a supportive AI assistant inside MentalOS. [...] Be supportive, calm and helpful. Always respond in the same language the user writes in."`
- Opcjonalnie `PersonalityHint` z frontendu → dodawany do systemu
- Wywołuje `ContextBuilder.BuildContextAsync(userId)` przed każdą odpowiedzią

OpenAiChatService

Klient HTTP wysyłający żądania do `{OpenAI:BaseUrl}chat/completions` z modelem `gpt-4o-mini`. Zwraca tekst pierwszej odpowiedzi.

OpenAiSpeechService

Wysyła plik audio jako `multipart/form-data` do endpointa Whisper. Model `gpt-4o-mini-transcribe`. Zwraca `TranscriptionResultDto { Success, Text, Error }`.

ContextBuilder

Buduje kontekst dla AI na podstawie danych użytkownika. Zawiera:

1. **Profil OCEAN** — wszystkie 5 wymiarów z poziomem (`high/medium/low`) i wynikiem numerycznym
2. **Ostatnie 3 wpisy dziennika** — treść skrócona do 120 znaków + tagi emocji
3. **Analiza nastroju z 7 dni** — średnia nastroju i top 3 dominujące emocje
4. **Zadania na dziś** — do 5 zadań z tytułem i statusem (`done/pending`)
5. **Produktywność** — `completed X/Y tasks today`

Anonimizacja przed wysłaniem do OpenAI:

- Adresy email → `[EMAIL_UKRYTY]`
- Numery 9–11 cyfr (PESEL, telefon) → `[NUMER_UKRYTY]`

StreakService

- `HandleDailyActivity(userId)` — sprawdza czy jest nowy wpis dziennika od ostatniej akcji streaku. Jeśli tak: `streak_count++`, `coins += streak_count` (nagroda narastająca — dzień 1 = 1 moneta, dzień 2 = 2 monety, itp.), `has_pending_fruit = true`.
- `CheckStreak(userId)` — jeśli brak aktywności dziś i wczoraj → reset: `streak_count = 0`, `streak_active = false`.

- `Add(userId, amount, action)` — ręczne dodanie monet z rejestracją w `streak_history`.

GardenService

Zarządza ogrodem składającym się z siatki **2×3 = 6 pól** (`GardenBed`).

Stany rośliny (`TreeState`): `Empty` → `Sprout` → `Sapling` → `Mature` → `Fruiting`

Czas przejścia między stanami:

- **Normalny tryb**: 1 dzień na etap (czas w dniach)
- **FastMode** (dev, konfigurowalny): `Garden:MinutesPerStage` minut na etap

Operacje:

- `PlantTreeAsync` — wymaga `FruitsBalance >= 1`; sadzi roślinę, odejmuje 1 owoc
- `HarvestTreeAsync` — wymaga `TreeState == Fruiting`; daje **30 monet** (przez `StreakService.AddBalance`)
- `ExchangeFruitAsync` — 1 owoc → 10 monet; raz dziennie (sprawdza `streak_history` z `action="fruit_action"`)

Ogród tworzony automatycznie przy pierwszym pobraniu (`GetGardenStatusAsync`).

ShopService

- `PurchaseTransaction` — waliduje saldo (`coins >= price`), zapisuje `UserItem`, rejestruje `CoinTransaction` z `type="expense"`
- `EquipItem` — oznacza jeden `UserItem` jako `IsActive=true`, pozostałe `IsActive=false`
- `UnequipItem` — ustawia `IsActive=false` na danym przedmiocie
- `PutShopItem` — upsert przedmiotu (create lub update po `FrontendAccesoriesId`)
- Sklep obsługuje `FrontendAccesoriesId` — gdy frontend przesyła ID string zamiast UUID, system szuka po tym polu i może auto-tworzyć przedmiot

PersonalityService (Big Five OCEAN)

Oblicza wyniki dla 5 wymiarów O/C/E/A/N na podstawie odpowiedzi z kwestionariusza. Zwraca słownik `{ "O": 3.8, "C": 4.2, ... }`. Pytania dostarczane przez `JsonQuestionProvider` z pliku JSON.

WelcomeRewardService

Jednorazowe nagrody za pierwsze wykonanie aktywności:

Typ nagrody	Monety	Trigger
"note"	10 monet	Pierwszy wpis w dzienniku z <code>preview</code>
"task"	5 monet	Pierwsze ukończone zadanie
"video"	10 monet	Pierwsze obejrzenie materiału wideo
"quiz"	50 monet	Pierwsze ukończenie quizu osobowości

Sprawdza flagi `HasReceived*Reward` w encji `User`. Po przyznaniu: ustawia flagę, dodaje monety, rejestruje `CoinTransaction` (`type="income", reason="welcome_reward_<type>"`).

PasswordResetService

1. `RequestResetAsync(email)` — generuje kryptograficznie bezpieczny token, zapisuje jego hash SHA-256 w `password_reset_tokens` z wygaśnięciem 24h, zwraca raw token
2. Link w emailu: `mentalos://reset-password?token=<raw_token>` (deep link)
3. `ResetPasswordAsync(token, newPassword)` — weryfikuje hash, sprawdza `ExpiresAt` i `UsedAt`, waliduje siłę hasła, hashuje nowe hasło, ustawia `UsedAt`

Rzucane wyjątki (obsługiwane w kontrolerze):

- `ExpiredResetTokenException` → HTTP 410
- `WeakPasswordException` → HTTP 400
- `NotLocalAccountException` → HTTP 400 (konto OAuth nie ma lokalnego hasła)
- `InvalidResetTokenException` → HTTP 400

GmailSmtpEmailService

Gmail SMTP przez MailKit. Port 587, TLS. Konfiguracja w sekcji `Smtp` w `appsettings.json`.

DailySummaryNotificationService i DataArchivingService

Hosted services (`IHostedService`) działające w tle niezależnie od obsługi requestów API.

8. Backend — kontrolery i endpointy API

Baza URL: `http://<host>:5076/api`

Swagger UI (tylko Development): `http://localhost:5076/swagger`

Nagłówek autoryzacji dla chronionych endpointów:

```
Authorization: Bearer <JWT_token>
```

AuthController — `/api/auth`

Metoda	Endpoint	Opis	Auth
POST	<code>/register</code>	Rejestracja (email + hasło)	✗
POST	<code>/login</code>	Logowanie lokalne	✗
POST	<code>/google</code>	OAuth Google (idToken)	✗
POST	<code>/facebook</code>	OAuth Facebook (accessToken)	✗
POST	<code>/forgot-password</code>	Żądanie resetu hasła (email → link)	✗

Metoda	Endpoint	Opis	Auth
POST	/reset-password	Ustawienie nowego hasła (token + newPassword)	✗

POST [/api/auth/register](#)

```
Request: { "email": "user@example.com", "password": "Pass123!",
"personalityType": "balanced" }
Response: { "token": "eyJ...", "user": { id, email, firstName, lastName, avatar,
streakCount, streakActive, coinsBalance, fruitsBalance, isPremium,
isAdmin, createdAt } }
```

POST [/api/auth/login](#)

```
Request: { "email": "...", "password": "..." }
Response 200: jak register
Response 401: { "message": "Invalid credentials" }
Response 400: { "message": "This account is registered with google. Please use
google login." }
```

POST [/api/auth/reset-password](#)

```
Request: { "token": "<raw_token>", "newPassword": "NewPass123!" }
Response 200: { "message": "Password has been changed." }
Response 410: { "error": "RESET_TOKEN_EXPIRED" }
Response 400: { "error": "WEAK_PASSWORD" | "INVALID_RESET_TOKEN" |
"NOT_LOCAL_ACCOUNT" }
```

UserController — [/api/users](#) (wymaga JWT)

Metoda	Endpoint	Opis
GET	/me	Profil zalogowanego użytkownika
PUT	/me	Aktualizacja firstName, lastName, avatar (URL)
POST	/me/premium	Aktywacja premium (flaga <code>IsPremium = true</code>)
DELETE	/me/premium	Anulowanie premium
DELETE	/me	Trwałe usunięcie konta (+ plik awatara z dysku)
POST	/avatar	Upload awatara (multipart/form-data, max 5 MB, .jpg/.png/.webp)

DELETE [/api/users/me](#) — wykonuje ręczne usunięcie kaskadowe:

1. Usuwa plik awatara z [wwwroot/avatars/](#)

2. Usuwa `ChatSessions` (wiadomości kaskadują z sesji)
3. Usuwa `UserItems`, `CoinTransactions`, `StreakHistories`, `Gardens`
4. Usuwa użytkownika (kaskada: `PersonalityProfile`, `UserRoles`, `PasswordResetTokens`, `JournalEntries`, `PlannerTasks`)

POST `/api/users/avatar`

- Dozwolone typy: `.jpg`, `.jpeg`, `.png`, `.webp`
- Limit: 5 MB
- Plik zapisywany w `wwwroot/avatars/<GUID><ext>`
- Stary awatar usuwany z dysku
- Zwraca: `{ "avatar": "/avatars/...", "avatarUrl": "http://..." }`

AdminController — `/api/admin` (wymaga roli `admin`)

Metoda	Endpoint	Opis
GET	<code>/users</code>	Lista wszystkich użytkowników z flagą <code>isAdmin</code>
DELETE	<code>/users/{id}</code>	Usuń użytkownika (400 jeśli ma rolę admin)
GET	<code>/logs?lines=100</code>	Ostatnie N linii z aktualnego pliku logów Serilog
POST	<code>/add-item</code>	Dodaj nowy przedmiot do sklepu (<code>CreateShopItemDto</code>)

GET `/api/admin/logs` zwraca:

```
{ "fileName": "log-2026-05-19.txt", "totalLines": 1250, "displayedLines": 100,
  "content": "..." }
```

JournalController — `/api/journal` (wymaga JWT)

Metoda	Endpoint	Opis
GET	<code>/</code>	Lista wszystkich wpisów (sortowanie: <code>created_at DESC</code>)
GET	<code>/date-range?startDate=&endDate=</code>	Wpisy z zakresu dat
GET	<code>/ {id}</code>	Konkretny wpis
POST	<code>/</code>	Utwórz wpis (+ streak + WelcomeReward jeśli pierwszy)
PUT	<code>/ {id}</code>	Aktualizuj wpis
DELETE	<code>/ {id}</code>	Usuń wpis (cofnięcie <code>HasPendingFruit</code> jeśli brak kwalifikujących wpisów)
GET	<code>/summary/{date}</code>	Statystyki dnia: <code>entryCount</code> , <code>averageMoodScore</code> , lista wpisów

Metoda	Endpoint	Opis
POST	<code>/daily-summary/generate</code>	Generuj AI-podsumowanie z kwestionariusza → zapisz w <code>preview</code> istniejącego wpisu
POST	<code>/daily-summary/generate-text</code>	Jak wyżej, zwraca tylko <code>{ "text": "..."} </code> bez zapisu
POST	<code>/ai-entry</code>	Twórz wpis z transkrybowanego tekstu (AI generuje title, content, moodScore, emotions)

POST `/api/journal/daily-summary/generate`

Kontekst dla AI: typ osobowości + zadania dnia (z statusem) + średni nastrój z ostatnich 7 dni.

Warunek: musi istnieć co najmniej jeden nie-podsumowujący wpis na dany dzień.

Wynik zapisywany w polu `preview` najnowszego wpisu z danego dnia.

ChatController — `/api/chat` (wymaga JWT)

Metoda	Endpoint	Opis
POST	<code>/session</code>	Utwórz sesję czatu (body: tytuł opcjonalny)
POST	<code>/message</code>	Wyślij wiadomość (<code>{ sessionId, message, personalityHint? }</code>)
GET	<code>/history/{sessionId}</code>	Historia wiadomości sesji
GET	<code>/debug-context</code>	Wyświetl zbudowany kontekst AI (diagnostyczny)

AssistantController — `/api/assistant` (wymaga JWT)

Metoda	Endpoint	Opis
POST	<code>/chat</code>	Wyślij wiadomość (auto-wybór lub tworzenie sesji)

Różnica od `ChatController`: `AssistantController` automatycznie wybiera lub tworzy sesję dla użytkownika — frontend nie musi zarządzać `sessionId`. Zwraca: `{ "reply": "<odpowiedź AI>" }`.

PlannerController — `/api/planner` (wymaga JWT)

Metoda	Endpoint	Opis
GET	<code>/</code>	Wszystkie zadania użytkownika (sortowanie po <code>task_date ASC</code>)
GET	<code>/daily?date=</code>	Zadania na konkretny dzień
GET	<code>/weekly?startDate=</code>	Zadania na tydzień (7 dni od startDate)
GET	<code>/monthly?year=&month=</code>	Zadania na miesiąc

Metoda	Endpoint	Opis
GET	<code>/{{id}}</code>	Konkretne zadanie
POST	<code>/</code>	Utwórz zadanie
PUT	<code>/{{id}}</code>	Aktualizuj zadanie (w tym <code>IsCompleted</code>)
PATCH	<code>/{{id}}/complete? isCompleted=</code>	Przełącz status ukończenia (+ <code>WelcomeReward</code> przy pierwszym)
DELETE	<code>/{{id}}</code>	Usuń zadanie

QuizController — `/api/quiz` (wymaga JWT)

Uwaga: Ten kontroler obsługuje uproszczony quiz typologiczny (4 typy). Nie korzysta z `PersonalityService`.

Metoda	Endpoint	Opis
GET	<code>/questions</code>	8 pytań z 4 odpowiedziami (A=empatyk, B=analitik, C=lider, D=marzyciel)
POST	<code>/submit</code>	Prześlij odpowiedzi → zlicz głosy → wyznacz dominujący typ → zapis w DB
GET	<code>/result</code>	Aktualny wynik (typ + tytuł + opis + punktacja)

Typy osobowości i ich opisy:

- **Empatyk** — ciepły, wrażliwy, słucha i wspiera innych
- **Analitik** — logiczny, precyzyjny, analizuje przed działaniem
- **Lider** — zdecydowany, energiczny, przejmuje inicjatywę
- **Marzyciel** — kreatywny, intuicyjny, otwarty na zmiany

Wynik QuizController: `{ "PersonalityType": "empatyk", "Title": "Empatyk", "Description": "...", "Scores": {"empatyk": 5, "analitik": 2, ...} }`

PersonalityController — `/api/personality` (wymaga JWT)

Uwaga: Oddzielny od QuizController — obsługuje model Big Five OCEAN.

Metoda	Endpoint	Opis
GET	<code>/questions</code>	Pytania Big Five (z <code>JsonQuestionProvider</code>)
GET	<code>/profile</code>	Wyniki OCEAN <code>{ "O": 3.8, "C": 4.2, "E": 2.5, "A": 4.0, "N": 2.1 }</code>
POST	<code>/submit</code>	Prześlij odpowiedzi → <code>PersonalityService.Calculate()</code> → zapis profilu

Wynik PersonalityController zawiera dodatkowo `welcomeReward` (za pierwsze ukończenie).

StreakController — `/api/streak` (wymaga JWT)

Metoda	Endpoint	Opis
POST	<code>/daily</code>	Obsługa codziennej aktywności (aktualizacja streak)
POST	<code>/add?amount=&action=</code>	Ręczne dodanie monet i wpisu do historii
GET	<code>/daily-status</code>	Pełny stan dzienny użytkownika
POST	<code>/claim-daily-reward?rewardType=</code>	Odbiór dziennej nagrody (<code>coins</code> lub <code>fruit</code>)
POST	<code>/claim-fruit</code>	Odbiór oczekującego owocu (<code>HasPendingFruit</code>)
GET	<code>/current-streak</code>	Aktualny licznik streak
POST	<code>/debug/add-fruits?amount=5</code>	Tylko Development — ręczne dodanie owoców

GET `/api/streak/daily-status` zwraca:

```
{
  "hasJournalEntry": true,      // czy jest wpis w dzienniku dziś
  "hasDaySummary": true,       // czy jest podsumowanie AI dziś
  "progress": 2,               // 0/1/2 (wpis + podsumowanie)
  "fruitsBalance": 3,
  "hasPendingFruit": false,    // czy czeka owoc do odebrania
  "streakCount": 7,           // aktualny streak (obliczany z wpisów dziennika)
  "coinsBalance": 150,
  "hasDailyFruitUsed": false,  // czy użyto wymiany owoc→monety dziś
  "hasDailyRewardClaimed": true // czy odebrano dzienną nagrodę
}
```

Logika streak: Streak obliczany na podstawie wpisów dziennika (`journal_entries`), nie `streak_history`.
Liczone są kolejne dni wstecz, w których użytkownik miał co najmniej jeden wpis (nie-podsumowanie).

POST `/api/streak/claim-daily-reward?rewardType=coins` — dodaje 10 monet

POST `/api/streak/claim-daily-reward?rewardType=fruit` — dodaje 1 owoc

Warunek: musi być wpis w dzienniku dziś; nagroda raz dziennie.

ShopController — `/api/shop` (wymaga JWT)

Metoda	Endpoint	Opis
GET	<code>/items</code>	Lista wszystkich aktywnych przedmiotów
POST	<code>/buy?itemId=</code>	Kup przedmiot (po UUID lub <code>FrontendAccesoriesId</code>); auto-tworzy jeśli nie istnieje
GET	<code>/purchase-history</code>	Historia transakcji monet użytkownika
GET	<code>/my-items</code>	Zakupione przedmioty z <code>FrontendAccesoriesId</code> i <code>IsActive</code>
POST	<code>/equip-item?itemId=</code>	Ekwipuj przedmiot (tylko jeden aktywny na raz)

Metoda	Endpoint	Opis
POST	<code>/unequip-item?itemId=</code>	Zdejmij przedmiot

Mechanizm `FrontendAccessoriesId`: Jeśli frontend przesyła `itemId` jako string (nie UUID) i przedmiot nie istnieje w bazie, endpoint `/buy` może auto-tworzyć go na podstawie danych z body (`CreateShopItemDto`). Umożliwia to pracę ze statycznymi "mockami" po stronie frontendu.

GardenController — `/api/garden` (wymaga JWT)

Trasa: `[Route("api/[controller]")] → /api/garden`

Metoda	Endpoint	Opis
GET	<code>/garden</code>	Stan ogrodu (lista pól z <code>TreeState</code> i pozycją X/Y) + <code>FruitsBalance</code>
POST	<code>/garden/plant/{gardenBedId}</code>	Zasadź roślinę (wymaga 1 owoc w saldzie)
POST	<code>/garden/harvest/{gardenBedId}</code>	Zbierz dojrzałą roślinę (+30 monet); zwraca <code>{ coinsBalance }</code>
POST	<code>/garden/exchange-fruit</code>	Wymień 1 owoc na 10 monet (raz dziennie)

MentalSupportController — `/api/mental-support` (wymaga JWT)

Metoda	Endpoint	Opis
POST	<code>/video-watched</code>	Rejestruje pierwsze obejrzenie wideo → zwraca <code>WelcomeReward</code> (10 monet)

Ten kontroler jest bardzo prosty — całość logiki materiałów wsparcia (lista medytacji, dźwięków, treningów) jest obsługiwana po stronie frontendu ze statycznych danych.

SpeechController — `/api/speech`

Metoda	Endpoint	Opis
POST	<code>/transcribe</code>	Transkrypcja audio (multipart/form-data)

Brak `[Authorize]` — endpoint bez wymogu JWT.

Request: `multipart/form-data` z polami `File` (`IFormFile`) i `language` (opcjonalne, np. "pl").

Response: `{ "Success": true/false, "Text": "...", "Error": "..." }`

9. Backend — middleware, DTOs i konfiguracja startowa

ErrorHandlingMiddleware

Przechwytuje wszystkie nieobsłużone wyjątki, loguje przez Serilog (`_logger.LogError`), zwraca:

```
{ "statusCode": 500, "message": "Internal Server Error", "detailed": "  
<exception.Message>" }
```

Rejestrowana jako **pierwszy middleware** w `Program.cs`.

Kolejność middleware pipeline

```
ErrorHandlingMiddleware (pierwszy – łapie wszystkie wyjątki)  
↓  
Swagger UI (tylko Development)  
↓  
CORS (AllowAnyOrigin / AllowAnyMethod / AllowAnyHeader)  
↓  
StaticFiles (wwwroot/ – awatary pod /avatars/)  
↓  
Antiforgery (wymagane przez Blazor SSR)  
↓  
Authentication (JWT Bearer)  
↓  
Authorization  
↓  
MapControllers + MapRazorComponents (Blazor)
```

Inicjalizacja przy starcie (Program.cs)

1. `db.Database.Migrate()` → automatyczne migracje
2. `ALTER TABLE users ADD COLUMN IF NOT EXISTS ...` → bezpieczne dodanie kolumn
(`fruits_balance`, `has_pending_fruit`, `has_received_*_reward`)
3. Sprawdź/utwórz rolę "admin"
4. Sprawdź/utwórz konto `admin@local` / `Admin123!`

Kluczowe DTOs

UserDataDto (odpowiedź przy logowaniu i `/me`):

```
{ "id", "email", "firstName", "lastName", "avatar", "streakCount", "streakActive",  
  "coinsBalance", "fruitsBalance", "isPremium", "isAdmin", "createdAt" }
```

WelcomeRewardDto (opcjonalnie dołączana do odpowiedzi):

```
{ "granted": true, "coinsAwarded": 10, "rewardType": "note" }
```

```
JournalEntryDto: { id, title, content, preview, moodScore, emotions, isSummary, entryDate,
createdAt, updatedAt }
```

```
PlannerTaskDto: { id, title, description, taskDate, hasTime, reminderTime, icon, category,
priority, recurrence, isCompleted, completedAt, createdAt, updatedAt }
```

```
GardenStatusDto: { "beds": [{ id, treeState, x, y }], "fruitsBalance": N }
```

10. Frontend — architektura i nawigacja

Routing (Expo Router — file-based)

Expo Router mapuje strukturę plików `app/` na trasy. Kluczowe trasy:

<code>/</code>	→ guard: <code>isAuthenticated</code> → <code>/(tabs)/home</code> <code>/login</code>
<code>/login</code>	→ ekran logowania
<code>/register</code>	→ ekran rejestracji
<code>/onboarding</code>	→ konfiguracja po rejestracji
<code>/psychotype</code>	→ quiz typologiczny (4 typy)
<code>/garden</code>	→ wirtualny ogród
<code>/accessories</code>	→ sklep / akcesoria
<code>/(tabs)/home</code>	→ ekran główny (Drzewo Dnia, Wrocznia, kafelki)
<code>/(tabs)/planer</code>	→ planer zadań
<code>/(tabs)/diary/</code>	→ dziennik (lista, edytor, nota, test dzienny)
<code>/(tabs)/mentalSupport/</code>	→ hub wsparcia (oddechy, medytacje, natura, treningi, video)
<code>/(tabs)/assistant</code>	→ asystent AI
<code>/(tabs)/profile</code>	→ profil użytkownika
<code>/psychologists/</code>	→ lista i profil psychologów (wymaga premium)
<code>/visits/</code>	→ historia wizyt

Drzewo komponentów (Root Layout `app/_layout.tsx`)

```
_layout.tsx
├─ AppInit (hydrate() → wczytanie tokenu z SecureStore)
├─ QueryClientProvider (TanStack React Query)
├─ GestureHandlerRootView
├─ Stack (Expo Router navigator)
│   └─ (tabs)/_layout.tsx → BottomTabBar (5 zakładek)
│       └─ zakładki: Planer | Dziennik | Home | Mental Support | Asystent
│           └─ pozostałe trasy (login, register, garden, itp.)
├─ AppToast (globalny system powiadomień)
└─ WelcomeRewardModal (modal animowanych nagród)
```

Ekran główny (home.tsx) — szczegóły

Drzewo Dnia — karta pokazująca postęp dzienny:

- Pobiera `GET /api/streak/daily-status` co 30 sekund (useFocusEffect + setInterval)
- Wizualizacja: tree-stage-0 (brak wpisu) → tree-stage-1 (wpis) → tree-stage-2 (wpis + podsumowanie, z jabłkiem)
- Jeśli `isReady` (hasJournalEntry && !hasDailyRewardClaimed) → przycisk "Odbierz 🍏"
- Modal wyboru nagrody: **jabłko** 🍏 (POST `/streak/claim-daily-reward?rewardType=fruit`) lub **10 monet** 🪙 (`coins`)

Wyrocznia — interaktywna funkcja chmurki:

- Kliknięcie kafelka "Wszechświat" → modal z animowaną chmurką
- Użytkownik zadaje pytanie w myślach i klika chmurkę
- Animacja "myślenia" (1400 ms) → losowa odpowiedź: **"Tak"** lub **"Nie"**
- Chmurka wyświetla zakupiony przez użytkownika awatar (zamiast domyślnej chmurki)

Kafelki na ekranie:

1. Wszechświat (Wyrocznia)
2. Akcesoria (→ `/accessories`)
3. Ogródek (→ `/garden`)

Premium Paywall:

- Kliknięcie "Praca z psychologiem" przez użytkownika bez premium → modal paywall
- Cena: 29 zł / miesiąc
- Po zakupie → redirect do `/psychologists`

11. Frontend — moduły funkcjonalne

Moduł Dziennika (`src/modules/diary/`)

Architektura offline-first z synchronizacją do serwera:

```
Użytkownik pisze tekst (DiaryEntryScreen)
  | router.replace('/diary/note', { text })
  ▼
DiaryNoteScreen: MoodSelector + TagSelector + SummaryInput
  | opcjonalnie: POST /api/journal/daily-summary/generate-text
  | → AI generuje podsumowanie na podstawie treści + kontekstu
  ▼
handleSave() → useDiaryEntries.addEntry(entry)
  |→ diaryService.create() → SQLite (sync_status="pending")
  |→ POST /api/journal → serwer
    ↓ response
    sync_status="synced", serverId=id
```

Lokalna tabela SQLite (diary.db):

```
CREATE TABLE diary_entries (  
  id TEXT PRIMARY KEY,  
  user_id TEXT NOT NULL,  
  content TEXT NOT NULL, -- serializowany JSON całego wpisu  
  sync_status TEXT DEFAULT 'pending', -- pending | synced | failed  
  server_id TEXT,  
  updated_at TEXT NOT NULL  
);
```

Podjęcie JSON-in-column dla `content` — dodanie nowego pola nie wymaga migracji SQLite.

Typ DiaryEntry (TypeScript):

```
type DiaryEntry = {  
  id: string  
  userId: string  
  title?: string  
  content: string  
  preview?: string // AI-generowane podsumowanie  
  moodScore?: number // 1-10  
  emotions?: string // JSON string '["Spokój", "Radość"]'  
  isSummary?: boolean  
  entryDate: string  
  createdAt: string  
  updatedAt: string  
  serverId?: string // ID z serwera po synchronizacji  
  syncStatus: "pending" | "synced" | "failed"  
}
```

Główne komponenty UI:

- `DiaryScreen` — lista z wyszukiwaniem, `useFocusEffect` odświeża po powrocie
- `DiaryEntryScreen` — edytor (react-native-pell-rich-editor), przycisk transkrypcji mowy
- `DiaryNoteScreen` — `MoodSelector` (1-10), `TagSelector` (emocje), `SummaryInput` (AI)
- `DiaryTestScreen` — dzienny kwestionariusz (5 pytań), wywołuje `POST /journal/daily-summary/generate`
- `DiaryEntryCard` — karta wpisu na liście (nastrój, tagi, fragment treści)

Moduł Asystenta AI (`src/modules/assistant/`)

Przepływ konwersacji:

1. Ekran `/assistant` → lista sesji czatu (`GET /chat/history/{sessionId}`)
2. Przycisk "Nowa rozmowa" → `POST /chat/session`
3. Wpisanie wiadomości → `POST /chat/message { sessionId, message, personalityHint? }`

4. Backend buduje kontekst i wywołuje GPT-4o-mini
5. Odpowiedź wyświetlana w `MessageBubble`

Alternatywny endpoint: `POST /assistant/chat { message, personalityHint? }` — auto-zarządzanie sesją.

Moduł Planera (`src/modules/planner/`)

Widoki: dzienny (kalendarz dzienny), tygodniowy, miesięczny.

Tworzenie zadania: title, description, data/czas (DateTimePicker), ikona, kategoria, priorytet, cykliczność.

`PATCH /{id}/complete` — przy pierwszym ukończeniu wyzwala `WelcomeReward` (5 monet).

Komponenty: `PlannerScreen`, `PlannerTaskCard`, `PlannerInputBar`, modale (data, czas, kategoria, notatki).

Moduł Ogrodu (`src/modules/garden/`)

Siatka 2×3 (6 pól `GardenBed`). Każde pole animowane przez `react-native-reanimated`.

Przepływ:

1. `GET /garden` → pobiera stan wszystkich pól
 2. Wolne pole (`TreeState.Empty`) → kliknięcie → `POST /garden/plant/{id}` (kosztuje 1 owoc)
 3. Po czasie (1 dzień normalny / konfigurowalny `FastMode`) pole dojrzewa przez: `Sprout` → `Sapling` → `Mature` → `Fruiting`
 4. `Fruiting` → kliknięcie → `POST /garden/harvest/{id}` (+30 monet)
 5. Przycisk "Wymień owoc" → `POST /garden/exchange-fruit` (1 owoc → 10 monet, raz dziennie)
-

Moduł Sklepu (`src/modules/shop/`)

Waluty: Monety (zdobywane za aktywność) i Owoce (z dziennych nagród i zbioru ogrodu).

Przedmioty: awatary/skórki chmurki — zmieniane przez `POST /shop/equip-item`.

Ekwipowanie aktualizuje `useShopStore.equippedPreviewImage` → zmiana chmurki na ekranie głównym i w Wyrocni.

Moduł Mental Support (`src/modules/mentalSupport/`)

Ekran: Medytacje, Oddychanie, Natura, Treningi, Wideo.

Materiały wideo: dynamiczne trasy `/mentalSupport/video/[id]` (`expo-video`).

Przy pierwszym obejrzeniu wideo: `POST /mental-support/video-watched` → `WelcomeReward` (10 monet).

Moduł Testu Osobowości (`src/modules/psychotype/`)

Obsługuje **quiz typologiczny** (`QuizController`, 8 pytań, 4 typy).

Pytania ze statycznej listy w `QuizController` (nie API).

Wynik wyświetlany z animowaną grafiką na ekranie `/psychotype`.

Po ukończeniu: `WelcomeReward` (50 monet przez `PersonalityController` lub `QuizController`).

12. Frontend — zarządzanie stanem

useAuthStore

Stan autoryzacji z trwałością w `expo-secure-store`:

```
interface AuthState {
  token: string | null
  user: UserPayload | null // { id, email, isPremium, isAdmin, coinsBalance,
  fruitsBalance, ... }
  isAuthenticated: boolean
  isLoading: boolean // true podczas hydrate(), false po zakończeniu

  login(token, user): Promise<void> // zapis + reset shop/visits +
  router.replace("/")
  loginSilent(token, user): Promise<void> // jak login, bez nawigacji (OAuth)
  logout(): Promise<void> // czyszczenie +
  router.replace("/login")
  hydrate(): Promise<void> // wczytanie z SecureStore przy starcie
  aplikacji
  buyPremium(): Promise<void> // POST /users/me/premium + update store
  cancelPremium(): Promise<void> // DELETE /users/me/premium + update
  store
}
```

Logika `hydrate()` (zaawansowana):

1. Wczytuje token i dane użytkownika z SecureStore
2. Dekoduje JWT przez `jwtDecode` i sprawdza `exp`
3. Jeśli token **wygasły** i serwer **dostępny** → wylogowuje
4. Jeśli token **wygasły** ale serwer **niedostępny** → wpuszcza (tryb offline grace period — sesja tymczasowo aktywna)
5. Jeśli token **ważny** → ustawia sesję, ładuje historię wizyt

useShopStore

Stan sklepu: lista zakupionych przedmiotów, aktywny przedmiot (`equippedPreviewImage`).

`fetchEquippedItem()` pobiera aktywny przedmiot z `GET /shop/my-items`.

useVisitStore

Historia wizyt u specjalistów. `loadForUser(userId)` ładuje per użytkownik z AsyncStorage lub serwera.

useRewardModalStore

Kontroluje `WelcomeRewardModal`. Aktywowany gdy odpowiedź backendu zawiera `welcomeReward.granted = true`.

useToastStore

`show(title, message, type)` — wyświetla powiadomienie Toast (`success` / `error` / `info`).

13. Frontend — warstwa API i komunikacja

Klient Axios (`src/services/api/client.ts`)

```
const apiClient = axios.create({
  baseURL: process.env.EXPO_PUBLIC_API_URL
  ?? (Platform.OS === 'android' ? 'http://10.0.2.2:5076/api'
    : 'http://172.20.10.3:5076/api'),
  timeout: 10_000,
});

// Interceptor wychodzący – dokłada JWT
apiClient.interceptors.request.use(config => {
  const token = useAuthStore.getState().token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Interceptor przychodzący – auto-wylogowanie przy 401
apiClient.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) useAuthStore.getState().logout();
    return Promise.reject(err);
  }
);
```

Moduły API

Plik	Kluczowe funkcje
<code>auth.ts</code>	<code>login()</code> , <code>register()</code> , <code>googleLogin()</code> , <code>facebookLogin()</code>
<code>diaryApi.ts</code>	<code>getEntries()</code> , <code>createEntry()</code> , <code>updateEntry()</code> , <code>deleteEntry()</code> , <code>generateSummary()</code> , <code>generateSummaryText()</code> , <code>generateAiEntry()</code>
<code>streakApi.ts</code>	<code>getDailyStatus()</code> , <code>claimDailyReward(type)</code> , <code>claimFruit()</code> , <code>triggerDaily()</code> , <code>trackVideoWatched()</code>
<code>assistant.ts</code>	<code>createSession()</code> , <code>sendMessage()</code> , <code>getHistory()</code>

DailyStatus (typ z `streakApi.ts`)

```
type DailyStatus = {
  hasJournalEntry: boolean // wpis w dzienniku dziś
  hasDaySummary: boolean // podsumowanie AI dziś
  progress: number // 0 / 1 / 2
```

```

fruitsBalance: number
hasPendingFruit: boolean // owoc do odebrania
streakCount: number
coinsBalance: number
hasDailyFruitUsed: boolean // wymiana owoc→monety dziś
hasDailyRewardClaimed: boolean // dzienna nagroda odebrana
}

```

useNetworkStatus (src/hooks/useNetworkStatus.ts)

Wykrywa dostępność sieci przez [@react-native-community/netinfo](#). Używany przez [diarySyncService](#).

React Query (TanStack)

[useQuery](#) — pobieranie danych z automatycznym odświeżaniem.

[useMutation](#) — mutacje (logowanie, tworzenie wpisów) z [onSuccess/onError](#).

[useAuthMutations.ts](#) eksportuje [useLoginMutation\(\)](#) i [useRegisterMutation\(\)](#).

14. Schemat bazy danych

Diagram relacji

```

users (1) — (N) user_roles — (N:1) roles
|
|— (0..1) personality_profiles
|
|— (N) journal_entries
|
|— (N) planner_tasks
|
|— (N) chat_sessions — (N) chat_messages
|
|— (N) streak_history
|
|— (N) coin_transaction
|
|— (N) user_item — (N:1) ShopItems
|
|— (1) Gardens — (N) GardenBeds
|
|— (N) password_reset_tokens

```

Tabele — szczegółowy opis

Tabela	Kluczowe kolumny	Uwagi
users	id (PK), email (UNIQUE), password_hash, streak_count, coins_balance, fruits_balance,	Soft delete: deleted_at

Tabela	Kluczowe kolumny	Uwagi
	has_pending_fruit, is_premium, has_received*_reward, Provider, ProviderUserId	
roles	id (PK), name (UNIQUE)	Soft delete
user_roles	user_id (FK), role_id (FK), UNIQUE(user_id, role_id)	Soft delete
personality_profiles	user_id (UNIQUE FK), personality_type, traits (JSON)	Soft delete; Jeden profil per user
journal_entries	user_id (FK CASCADE), content (NOT NULL), mood_score (1–10, nullable), emotions (CSV), preview (AI), is_summary, entry_date	Hard delete (nie soft)
planner_tasks	user_id (FK CASCADE), title, task_date, priority (int enum), recurrence (int enum), is_completed	is_deleted + archived_at
chat_sessions	user_id (FK), title, created_at, updated_at	—
chat_messages	session_id (FK CASCADE), sender ("user"/"assistant"), content	—
streak_history	user_id (FK), date, action, streak_value, balance_after	action: "daily", "streak losted", "daily_reward_claimed", "fruit_action", "harvest"
coin_transaction	user_id (FK CASCADE), amount, reason, type ("income"/"expense"), balance_after	—
ShopItems	id, accesories_id (FrontendAccesoriesId), name, price, type, is_active	Brak FK do users
user_item	user_id (FK CASCADE), shop_item_id (FK CASCADE), is_active, UNIQUE(user_id, shop_item_id)	is_active — ekwipowany
Gardens	user_id (FK)	Auto-tworzony przy pierwszym GET
GardenBeds	garden_id (FK CASCADE), tree_state (enum), planted_at, X, Y, UNIQUE(garden_id, X, Y)	Siatka 2×3 = 6 pól
password_reset_tokens	user_id (FK CASCADE), token_hash varchar(64) (UNIQUE), expires_at, used_at	SHA-256, 24h ważność

15. Uwierzytelnienie i autoryzacja

Tokeny JWT

Parametr	Wartość
Algorytm	HMAC-SHA256
Issuer	MentalIOS (konfigurowalny)
Audience	MentalIOS.Clients (konfigurowalny)
Ważność	Jwt:ExpiryMinutes (domyślnie 1440 min = 24 h)
ClockSkew	30 sekund

Claims w tokenie: `ClaimTypes.NameIdentifier` (User GUID), `ClaimTypes.Email`, `ClaimTypes.Role` (osobny claim dla każdej roli).

Przechowywanie po stronie frontendu: `expo-secure-store` → Keychain (iOS) / Android Keystore (Android) → AES-256.

Mechanizm Offline Grace Period (frontend)

Jeśli przy starcie aplikacji token JWT wygaś, ale serwer jest niedostępny (`canReachBackend()` zwraca false) — aplikacja wpuszcza użytkownika mimo wygaśniętego tokenu. Sesja oznaczana jako tymczasowa (log ostrzeżenia). Przy kolejnym uruchomieniu z dostępnym serwerem — wymuszenie wylogowania.

OAuth 2.0

Google: `GoogleJsonWebSignature.ValidateAsync(idToken, new ValidationSettings { Audience = [GoogleClientId] })` → pobiera `Subject`, `Email`, `Name`.

Facebook: HTTP GET do Graph API → pobiera `id`, `email`, `name`.

W obu przypadkach: nowy użytkownik tworzony z `Provider="google"/"facebook"`, `ProviderUserId=externalId`, rola `"user"` przypisywana automatycznie.

Haszowanie haseł

ASP.NET Core `PasswordHasher<User>` (PBKDF2 + HMAC-SHA256, 10 000 iteracji, losowy salt). Wynik przechowywany w `password_hash`.

Autoryzacja w kontrolerach

```
[Authorize] // JWT wymagany
[Authorize(Roles = "admin")] // Wymaga roli "admin" w JWT claims
```

Ekstrakcja ID użytkownika z tokenu:

```
var userId = Guid.Parse(User.FindFirst(ClaimTypes.NameIdentifier)?.Value);
```

CORS

Aktualnie `AllowAnyOrigin()` — wymagane dla połączeń z aplikacji mobilnej. Przed wdrożeniem produkcyjnym należy ograniczyć do konkretnych domen.

16. Konfiguracja i zmienne środowiskowe

Backend — appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection":
    "Host=localhost;Port=5432;Database=mentalos_db;Username=mentalos;Password=mentalos_password"
  },
  "Jwt": {
    "Key": "<ZMIENÍ — minimum 32 znaki, kryptograficznie losowy>",
    "Issuer": "MentalOS",
    "Audience": "MentalOS.Clients",
    "ExpiryMinutes": 1440
  },
  "OAuth": {
    "Google": { "ClientId": "...", "ClientSecret": "..." },
    "Facebook": { "AppId": "...", "AppSecret": "..." }
  },
  "OpenAI": {
    "ApiKey": "<OpenAI API Key>",
    "Model": "gpt-4o-mini",
    "BaseUrl": "https://api.openai.com/v1/"
  },
  "Smtp": {
    "Host": "smtp.gmail.com",
    "Port": 587,
    "User": "<konto Gmail>",
    "Pass": "<hasło aplikacji Gmail — nie hasło konta>",
    "From": "MentalOS <noreply@example.com>"
  },
  "App": {
    "ResetPasswordUrlBase": "mentalos://reset-password?token="
  },
  "Garden": {
    "FastMode": false,
    "MinutesPerStage": 2
  }
}
```

Sekcja `Garden`: `FastMode: true` + `MinutesPerStage: 2` → roślina dojrzewa w minutach (tryb deweloperski).

Porty backendu

Protokół	Adres	Opis
HTTP	<code>http://0.0.0.0:5076</code>	Aktywny (nasłuch na wszystkich interfejsach)
HTTPS	<code>https://localhost:5078</code>	Wyłączony (<code>UseHttpsRedirection</code> zakomentowane)

Frontend — plik `.env`

Tworzony w katalogu `frontend/`:

```
# Android Emulator (AVD – 10.0.2.2 = localhost maszyny hosta):
EXPO_PUBLIC_API_URL=http://10.0.2.2:5076/api

# iOS Simulator:
# EXPO_PUBLIC_API_URL=http://127.0.0.1:5076/api

# Fizyczne urządzenie (ta sama sieć Wi-Fi):
# EXPO_PUBLIC_API_URL=http://<IP_KOMPUTERA>:5076/api

# Expo Go z tunelem (ngrok/etc.):
# EXPO_PUBLIC_API_URL=https://<subdomain>.ngrok.io/api
```

17. Przepływy danych — kluczowe scenariusze

Rejestracja i pierwsze logowanie

```
Formularz rejestracji (email + hasło)
|
▼ POST /api/auth/register
Backend: hash hasła (PBKDF2) → INSERT users → przypisanie roli "user"
|
▼ { token, user }
Frontend: useAuthMutations.login(token, user)
→ expo-secure-store.saveToken(token)
→ useAuthStore.isAuthenticated = true
→ router.replace("/")
|
▼ app/index.tsx guard: isAuthenticated=true → /(tabs)/home
|
▼ (opcjonalnie) Onboarding → /psychotype (quiz) → POST /quiz/submit
→ profil osobowości zapisany, WelcomeReward 50 monet
```

Zapis wpisu dziennika z AI-podsumowaniem

```
DiaryEntryScreen: użytkownik pisze tekst
| router.replace('/diary/note', { text })
▼
```

```

DiaryNoteScreen: MoodSelector + TagSelector
  | opcjonalnie: POST /journal/daily-summary/generate-text
  | Backend: kontekst (profil + zadania + historia nastroju) → GPT-4o-mini →
podsumowanie
▼
handleSave() → useDiaryEntries.addEntry()
  |→ SQLite INSERT (sync_status="pending")
  |→ POST /api/journal { content, preview, moodScore, emotions }
    Backend: INSERT journal_entries
    Backend: TryUpdateStreakAsync:
      hasContent AND hasPreview? → user.HasPendingFruit = true
    Backend: WelcomeRewardService.TryGrant("note") → +10 monet (1. raz)
    ↓ { id, ..., welcomeReward? }
    SQLite UPDATE: sync_status="synced", server_id=id
    WelcomeRewardModal: jeśli welcomeReward.granted → animacja nagrody

```

System streaku — mechanika dzienna

```

Użytkownik ma wpis dziennika z preview (podsumowaniem)
  |
  ▼ Backend: user.HasPendingFruit = true (ustawione w TryUpdateStreakAsync)
  |
  ▼ Ekran Home: loadDailyStatus() → { hasPendingFruit: true, progress: 2 }
    Drzewo Dnia → etap 2 (jabłko na drzewie)
    Przycisk "Odbierz 🍏" aktywny
  |
  ▼ POST /streak/claim-daily-reward?rewardType=fruit
    Backend:
      - Sprawdza: hasJournalEntry=true, alreadyClaimed=false
      - FruitsBalance += 1
      - HasPendingFruit = false
      - INSERT streak_history { action="daily_reward_claimed" }
  |
  ▼ Frontend: hasDailyRewardClaimed = true, fruitsBalance++
    Drzewo Dnia → "✓ Odebrano"

```

Wirtualny ogród — cykl rośliny

```

FruitsBalance >= 1 → kliknij wolne pole (TreeState.Empty)
  | POST /garden/plant/{gardenBedId}
  ▼ Backend: FruitsBalance -= 1, PlantedAt = now, TreeState = Sprout

Po 1 dniu (lub MinutesPerStage minutach w FastMode):
  Sprout → Sapling → Mature → Fruiting (po 3 dniach/etapach)

Fruiting → kliknij pole
  | POST /garden/harvest/{gardenBedId}
  ▼ Backend: CoinsBalance += 30, TreeState = Empty, PlantedAt = null

```

Alternatywnie: POST /garden/exchange-fruit
 | 1 owoc → 10 monet (raz dziennie)

Czat z asystentem AI

```
POST /chat/session → { sessionId }
|
▼ POST /chat/message { sessionId, message: "Jak radzić sobie ze stresem?" }
|
▼ Backend: ChatService.SendMessageAsync()
  1. Zapis wiadomości użytkownika w chat_messages
  2. ContextBuilder.BuildContextAsync(userId):
    - PersonalityProfile (OCEAN traits)
    - 3 ostatnie wpisy dziennika (skrótowe do 120 znaków, anonimizowane)
    - Średni nastrój + top 3 emocje z ostatnich 7 dni
    - Do 5 zadań na dziś z statusem
    - Stosunek ukończonych do zaplanowanych
  3. OpenAI API: system prompt + kontekst + ostatnie 20 wiadomości sesji
  4. Zapis odpowiedzi asystenta w chat_messages
|
▼ { "response": "<odpowiedź AI>" } → MessageBubble
```

18. Instalacja i uruchomienie

Wymagania wstępne

Narzędzie	Wersja	Zastosowanie
.NET SDK	8.0+	Backend
PostgreSQL	16+	Baza danych
Node.js	18+ LTS	Frontend
npm	9+	Zarządzanie paczkami
Android Studio lub Xcode	aktualne	Emulator / Symulator
Docker (opcjonalnie)	—	Konteneryzacja backendu

Backend — krok po kroku

```
# 1. Utwórz bazę danych PostgreSQL
psql -U postgres
CREATE DATABASE mentalos_db;
CREATE USER mentalos WITH PASSWORD 'mentalos_password';
GRANT ALL PRIVILEGES ON DATABASE mentalos_db TO mentalos;
\q
```

```
# 2. Skonfiguruj appsettings.json w backend/MentalOS/
#   - ConnectionStrings.DefaultConnection
#   - Jwt.Key (min. 32 znaki – ZMIEN PRZED PRODUKCJĄ)
#   - OpenAI.ApiKey (wymagane dla AI chat i transkrypcji)
#   - Smtplib.* (wymagane dla resetu hasła)
#   - OAuth.Google.*, OAuth.Facebook.* (opcjonalne)

# 3. Wejdź do katalogu projektu
cd backend/MentalOS

# 4. Przywróć paczki i zbuduj
dotnet restore
dotnet build

# 5. Uruchom
dotnet run
# Backend: http://0.0.0.0:5076
# Swagger: http://localhost:5076/swagger
# Migracje i admin@local uruchamiają się automatycznie przy starcie

# Ręczne migracje:
dotnet ef database update
```

Frontend — krok po kroku

```
# 1. Przejdź do katalogu frontend
cd frontend

# 2. Zainstaluj zależności
npm install

# 3. Utwórz plik .env
echo "EXPO_PUBLIC_API_URL=http://10.0.2.2:5076/api" > .env
# (dostosuj adres do konfiguracji sieci)

# 4. Uruchom serwer deweloperski
npx expo start -c

# Opcje uruchamiania:
# 'a' → Android Emulator (AVD)
# 'i' → iOS Simulator (macOS)
# 'w' → przeglądarka
# Kod QR → Expo Go na fizycznym urządzeniu

# Budowanie natywne (generuje plik APK/IPA):
npx expo run:android
npx expo run:ios
```

Typowe problemy i rozwiązania

Problem	Rozwiązanie
Nie można połączyć z PostgreSQL	Sprawdź, czy PostgreSQL działa; zweryfikuj dane w <code>appsettings.json</code>
Port 5432 zajęty	Zatrzymaj wbudowaną usługę PostgreSQL Windows lub zmień port
<code>JWT key too short</code>	Klucz musi mieć ≥ 32 znaki
HTTPS redirect error	<code>app.UseHttpsRedirection()</code> zakomentowane w Program.cs — OK
Frontend nie dociera do backendu	Sprawdź <code>EXPO_PUBLIC_API_URL</code> ; na emulatorze Android <code>10.0.2.2</code> zamiast <code>localhost</code>
<code>Cannot connect to Metro</code>	Upewnij się, że komputer i urządzenie są w tej samej sieci Wi-Fi
Błąd CORS	Backend ma <code>AllowAnyOrigin()</code> — powinno działać; sprawdź logi

Docker (backend)

```
# Buduj obraz
docker build -t mentalos-api ./backend/MentalOS

# Uruchom z przekazanymi zmiennymi środowiskowymi
docker run -p 5076:5076 \
  -e
  ConnectionStrings__DefaultConnection="Host=host.docker.internal;Port=5432;Database=mentalos_db;Username=mentalos;Password=mentalos_password" \
  -e Jwt__Key="your-32-char-minimum-secret-key" \
  -e OpenAI__ApiKey="sk-..." \
  mentalos-api
```

19. Logowanie i monitoring

Serilog

Backend loguje zdarzenia strukturalnie do konsoli i rotowanych plików dziennych.

```
Log.Logger = new LoggerConfiguration()
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.File("Logs/log-.txt", rollingInterval: RollingInterval.Day)
    .CreateLogger();
```

Lokalizacja: `backend/MentalOS/Logs/log-YYYY-MM-DD.txt`

Rotacja: Nowy plik każdego dnia.

Przykładowe wpisy:


```
2026-05-19 10:00:00 [INF] Application started on http://0.0.0.0:5076
2026-05-19 10:00:01 [INF] Database connection successful!
2026-05-19 10:00:02 [INF] Admin role already exists
2026-05-19 10:05:30 [INF] User user@example.com logged in via local
2026-05-19 10:06:15 [INF] User user@gmail.com logged in via Google
2026-05-19 10:10:00 [ERR] Error authenticating with Google: Invalid token
2026-05-19 11:00:00 [INF] Transcribe endpoint wywołany; plik: recording.m4a, 42000 bytes
```

Poziomy: **INF** (normalne), **WRN** (ostrzeżenia), **ERR** (błędy wymagające uwagi).

Dostęp do logów:

- Bezpośrednio: `backend/MentalIOS/Logs/`
- Przez API (tylko admin): `GET /api/admin/logs?lines=100`

20. Bezpieczeństwo

Zastosowane mechanizmy

Obszar	Implementacja
Hasła	PBKDF2 + HMAC-SHA256 + losowy salt (ASP.NET Core Identity PasswordHasher)
Tokeny JWT	HMAC-SHA256, 24h ważność, walidacja issuer/audience/lifetime/signing key
OAuth tokeny	Google: <code>GoogleJsonWebSignature.ValidateAsync</code> ; Facebook: Graph API
Storage tokenów	<code>expo-secure-store</code> (OS Keychain/Keystore, AES-256)
Tokeny resetu hasła	SHA-256 hash w bazie, raw token w emailu, 24h wygaśnięcie, jednorazowy
Auto-wylogowanie	Interceptor Axios: HTTP 401 → <code>useAuthStore.logout()</code>
Offline grace	Wygasły token zachowany tylko gdy serwer niedostępny
Avatar upload	Walidacja rozszerzenia (.jpg/.jpeg/.png/.webp) + limit 5 MB
AI anonimizacja	Email i numery telefonów/PESEL maskowane przed wysłaniem do OpenAI

Kwestie wymagające rozwiązania przed produkcją

- CORS** — zmienić `AllowAnyOrigin()` na konkretne domeny frontendu
- HTTPS** — odkomentować `app.UseHttpsRedirection()`, skonfigurować certyfikat SSL
- JWT Key** — zastąpić domyślny klucz losowym, bezpiecznym (≥ 32 znaki); użyć zmiennych środowiskowych
- Hasło admina** — zmienić `Admin123!` po pierwszym uruchomieniu
- Connection string** — przechowywać w zmiennych środowiskowych (nie w plikach konfiguracji)
- OpenAI API Key** — zmienne środowiskowe, nigdy w repozytorium
- SMTP hasło** — hasło aplikacji Gmail zamiast hasła do konta; zmienne środowiskowe

21. Znane ograniczenia i plany rozwoju

Aktualne ograniczenia

Ograniczenie	Opis
HTTPS wyłączone	Backend pracuje wyłącznie po HTTP; wymaga konfiguracji SSL dla produkcji
CORS otwarty	<code>AllowAnyOrigin</code> — do ograniczenia przed produkcją
Specjaliści — dane statyczne	Moduł <code>psychologists</code> i <code>visits</code> oparty na danych statycznych po stronie frontendu; brak backendu dla rezerwacji wizyt
Premium — brak bramki płatności	<code>IsPremium = true</code> ustawiany bez integracji z App Store / Google Play Billing
Powiadomienia push	Infrastruktura zaimplementowana (<code>DailySummaryNotificationService</code>), ale wymaga konfiguracji tokenów FCM/APNs
Cykliczność zadań	Enum <code>PlannerTaskRecurrence</code> zdefiniowany, ale logika generowania cyklicznych instancji zadań nie jest jeszcze w pełni wdrożona
Dwa systemy osobowości	<code>QuizController</code> (4 typy: empatyk/analitik/lider/marzyciel) i <code>PersonalityController</code> (Big Five OCEAN) zapisują do tej samej tabeli <code>personality_profiles</code> w różnych formatach <code>traits</code> — potencjalna niezgodność
Brak testów automatycznych	Brak zestawu testów jednostkowych i integracyjnych

Planowane funkcjonalności

- Pełna integracja powiadomień push (FCM dla Android, APNs dla iOS)
- Rezerwacja wizyt u specjalistów z backendem (harmonogram, potwierdzenia emailem)
- Zarządzanie subskrypcją premium przez natywne API płatności
- Zaawansowany panel analityczny dla użytkownika (wykresy nastroju, aktywności, produktywności)
- Eksport danych dziennika (PDF, CSV) — funkcja zgodności z RODO
- Unifikacja systemu osobowości (jeden format `traits` dla obu quizów)
- Implementacja cykliczności zadań (generowanie instancji przy zapytaniach o zakres dat)
- Wsparcie trybu ciemnego
- Widżety ekranu głównego

Zespół: X

Backend: ASP.NET Core 8 + PostgreSQL 16 | Frontend: React Native 0.81.5 + Expo 54 | AI: OpenAI GPT-4o-mini + Whisper